

RISC-V Tutorial

Sign/Zero Extension, Little/big-endian, and addressing modes in
RISC-V

Endianness

- The values stored in byte addresses 0 to 3 are FA, 78, 1B and C3. Write the 4B variable that is stored in this location in a
 - big-endian system and,
 - a little endian system.

Byte Address	Contents
0	FA
1	78
2	1B
3	C3

Endianness

- The values stored in byte addresses 0 to 3 are FA, 78, 1B, and C3. Write the 4B variable that is stored in this location in a
 - big-endian system → 0xFA781BC3
 - a little-endian system → 0xC31B78FA

Byte Address	Contents
0	FA
1	78
2	1B
3	C3

Sign-Extension

- We use sign extension. We store 0x9CAF in a 32b register. Show its value in hexadecimal.
- What if we don't use sign extension?

Sign-Extension

- We use sign extension. We store 0x9CAF in a 32b register. Show its value in hexadecimal.

0xFFFF9CAF, since the MSB of 0x9 is 1, so the top 16 bits get filled with 1s.

Sign extension and Load and store instructions.

- What will be the contents of the registers after running the following RISC-V instructions:

.data

value1: .word 0x12345678

value2: .word 0x12f4f688

.text

la x10, value1

lw x11, 0(x10)

lh x12, 0(x10)

lh x13, 2(x10)

lb x14, 0(x10)

lb x15, 1(x10)

lb x16, 2(x10)

lb x17, 3(x10)

lb x18, 4(x10)

la x10, value2

lw x11, 0(x10)

lh x12, 0(x10)

lh x13, 2(x10)

lb x14, 0(x10)

lb x15, 1(x10)

lb x16, 2(x10)

lb x17, 3(x10)

lb x18, 4(x10)

Load and store instructions.

- What will be the contents of the registers after running the following RISC-V instructions:

.data

value1: .word 0x12345678

value2: .word 0x12f4f688

.text

la x10, value

lw x11, 0(x10) # x11 <- 0x12345678

lh x12, 0(x10) # x11 <- 0x00005678

lh x13, 2(x10) # x11 <- 0x00001234

lb x14, 0(x10) # x11 <- 0x00000078

lb x15, 1(x10) # x11 <- 0x00000056

lb x16, 2(x10) # x11 <- 0x00000034

lb x17, 3(x10) # x11 <- 0x00000012

lb x18, 4(x10) # x11 <- 0x00000000

Load and store instructions.

- What will be the contents of the registers after running the following RISC-V instructions:

.data

value1: .word 0x12345678

value2: .word 0x12f4f688

.text

la x10, value2

lw x11, 0(x10)

x11 <- 0x12f4f688

lh x12, 0(x10)

x11 <- 0xffffffff688

lh x13, 2(x10)

x11 <- 0x000012f4

lb x14, 0(x10)

x11 <- 0xffffffff88

lb x15, 1(x10)

x11 <- 0xffffffff6

lb x16, 2(x10)

x11 <- 0xffffffff4

lb x17, 3(x10)

x11 <- 0x00000012

lb x18, 4(x10)

x11 <- 0xffffffff88

How do we address the operands in RISC-V?

- Addressing modes in RISC-V:
 - Register direct (also called register only)
 - Immediate
 - Base-offset addressing (or register indirect)
 - PC Relative

How do we address the operands in RISC-V?

- Identify the addressing modes in the following instructions:
 - add s0, t2, t3
 - addi s4, t5, -73
 - sub t6, s1, x0
 - lw s4, 72(x0)
 - ori t3, t7, 0xFF
 - sw t2, -25(t1)
 - bne s8, s9, L1
 - jal label

How do we address the operands in RISC-V?

- Identify the addressing modes in the following instructions:

- add s0, t2, t3 → Register only
- addi s4, t5, -73 → Immediate
- sub t6, s1, x0 → Register only
- lw s4, 72(x0) → Base addressing
- ori t3, t7, 0xFF → Immediate
- sw t2, -25(t1) → Base addressing
- bne s8, s9, L1 → PC-relative
- jal label → PC-relative

C to RISC-V (Arithmetic, Logical)

RISC-V

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:

`a <- x10, b <- x11 and c <- x12.`

`a = b + c;`

`a = b - c;`

Is there any special difference between the two instructions?

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:

$a \leftarrow x10, b \leftarrow x11$ and $c \leftarrow x12$.

$a = b + c;$ `add x10, x11, x12`

$a = b - c;$ `sub x10, x11, x12`

Is there any special difference between the two instructions?

- In the case of subtraction, the order of the two source registers matters.

What is the mistake in this?

Add a, b, c //Mistake: the RISC-V code should not use C variables, but only registers

Add x10, x11, -x12 //Mistake: you cannot put a minus in front of a register. That is wrong.

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:

$a \leftarrow x10, b \leftarrow x11, c \leftarrow x12, d \leftarrow x13, \text{ and } e \leftarrow x14.$

$a = b + c + d - e;$

$a = (b + c) - (d + e)$

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:
a <- x10, b <- x11, c <- x12, d <- x13, and e <- x14.

a = b + c + d - e;

add x10, x11, x12
add x10, x10, x13
sub x10, x10, x14

a = (b + c) - (d + e)

add x10, x11, x12
add x15, x13, x14
sub x10, x10, x15

What is the mistake in this code?

add x10, x11, x12
add x13, x13, x14
sub x10, x10, x13

Answer: This code modifies d variable (x13 register), which is not done in the original code.

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:

`f <- x10 and g <- x11.`

`f = g + 10;`

`f = g - 10;`

`f = 10 - g;`

* There is no 'subi' instruction.

What makes the last instruction different?

Arithmetic instructions.

- What are the corresponding RISC-V instructions for the following C-codes? Assume:

$f \leftarrow x_{10}$ and $g \leftarrow x_{11}$.

$f = g + 10;$ `addi x10, x11, 10`

$f = g - 10;$ `addi x10, x11, -10`

$f = 10 - g;$ `addi x10, x0, 10` `Addi x10 x11 -10`
 `sub x10, x10, x11` `sub x10 x0 x10`

What makes the last instruction different?

The last operation will require two assembly instructions instead of one in the previous two.

Logical instructions.

- If we have a number 0x12345678 in x11 and we want to only keep the LSB / MSB of the number, how will we do it? Hint: A logical instruction can be used.
- There is no logical “NOT” instruction in RISC-V. How will you perform a NOT operation, then?
- The register x11 = 0x00000002. What will be the contents after the following instruction is run?

`slli x11, x11, 2`

* What is this operation equivalent to?

Logical instructions.

- If we have a number 0x12345678 in x11 and we want to only keep the LSB / MSB of the number, how will we do it? Hint: A logical instruction can be used.

For LSB:

`andi x12, x11, 0xff`

For MSB:

`li x13, 0xff000000`

`and x14, x11, x13`

* These are known as masking operations.

- There is no logical “NOT” instruction in RISC-V. How will you perform a NOT operation, then?

* `xori x11, x11, -1`

- The register x11 = 0x00000002. What will be the contents after the following instruction is run?

`slli x11, x11, 2`

- What is this operation equivalent to?

* Multiplication by 2^n (n -> no. of shifts)

Logical operations.

- If the register contained the value $(-25)_{10}$. What will be the value after the following instruction is executed?

`srai x10, x10, 4`

* Is this equivalent to division by 2^n .

Logical operations.

- If the register contained the value $(-25)_{10}$. What will be the value after the following instruction is executed?

`srai x10, x10, 4`

- Is this equivalent to division by 2^n .
 - No, it's not always equivalent to division, unlike the left shift for multiplication. Right shifts as division do not work for odd negative numbers.

LUI and AUIPC

For questions in this PPT, do not use "li"
instruction.

LUI (Load Upper Immediate)

- **Format:** LUI rd, imm
- **Operation:** $rd = imm \ll 12$
- What would be loaded into the x5 register after the execution of the following instruction?
 - LUI x5, 0x12345
- Load the value 0x12345678 in x10.

LUI (Load Upper Immediate)

- **Format:** LUI rd, imm
- **Operation:** $rd = imm \ll 12$
- What would be loaded into the x5 register after the execution of the following instruction?
 - LUI x5, 0x12345 $\rightarrow$ # x5 = 0x12345000
- Load the value 0x12345678 in x10.
 - LUI x10, 0x12345 $\rightarrow$ # x10 = 0x12345000
 - ADDI x10, x10, 0x678 $\rightarrow$ # x10 = 0x12345000 + 0x678 = 0x12345678
- **Common use:** Building large constants by combining LUI with ADDI:

LUI	x5,	0x12345	#	x5 = 0x12345000
ADDI	x5,	x5,	0x678	# x5 = 0x12345678

LUI (Load Upper Immediate)

- Load the value 12345678 in x10.

LUI (Load Upper Immediate)

- Load the value 12345678 in x10.
 - Option 1: Directly work with decimal numbers
 - `lui x10, 3014`
 - `Addi x10 x10 334`
 - Option 2: Convert it into hexadecimal and use the previous approach.
 - This number is 0xBC614E in hexadecimal
 - `Lui x10, 0xBC6`
 - `Addi x10, x10, 0x14E`

Memory accesses using LUI

- Load a word from the absolute address 0x80000000 into the register x6.
- Store the value in x7 to the address location 0x80000100. Assume RV32

Memory accesses using LUI

- Load the value from the absolute address 0x80000000 into the register x6.

```
LUI x5, 0x80000    # x5 = 0x80000000
```

```
LW x6, 0(x5)       # x6 ← (0x80000000)
```

- Store the value in x7 to the address location 0x80000100.

```
LUI x5, 0x80000    # x5 = 0x80000000
```

```
ADDI x5, x5, 0x100  # x5 = 0x80000100
```

```
SW x7, 0(x5)        # (x7) → 0x80000100
```

AUIPC (Add Upper Immediate to PC)

- **Format:** AUIPC rd, imm
- **Operation:** $rd = PC + (imm \ll 12)$
- If the current value of the program counter is 0x10000, what will be the value of x5 after the execution of the following instruction?
 - AUIPC x5, 0x1

AUIPC (Add Upper Immediate to PC)

- **Format:** AUIPC rd, imm
- **Operation:** $rd = PC + (imm \ll 12)$
- If the current value of the program counter is 0x10000, what will be the value of x5 after the execution of the following instruction?
 - AUIPC x5, 0x1 $\rightarrow$ # x5 = 0x10000 (PC) + 0x1000 (0x1 $\ll$ 12)
= 0x11000
- **Common uses:**
 - **Position-independent code** - calculating addresses relative to PC
 - **Accessing global data** - combined with load/store instructions
 - **Long jumps** - combined with JALR for calls beyond $\pm 1\text{MB}$ range

AUIPC (Add Upper Immediate to PC)

- Access data that's 4KB ahead of current PC.
- 4K is 4096, which is 1 followed by 12 zeros

AUIPC (Add Upper Immediate to PC)

- Access data that's 4KB ahead of the current PC.

```
AUIPC x10, 0x1    # x10 = PC + 0x1000
LW x11, 0(x10)    # Load from PC + 0x1000
```

Another way to write the same code

```
AUIPC x10, 0      # Get PC value in x10
LUI x9, 0x1       # Load 4096 into x9
add x9, x9, x10    # add PC+4096
LW x11, 0(x10)    # Load from PC + 0x1000
```

Key Differences

Aspect	LUI	AUIPC
Base value	0	Current PC
Purpose	Load constants	PC-relative addressing
Result	Absolute value	PC-relative address

- Both instructions are essential for creating full 32-bit addresses and values.

Sign Extension, Load Store,

Sign/Zero Extension, Little/big-endian, and addressing modes in
RISC-V

Sign extension and Load and store instructions.

Assume RV32. Any location not written stores 0.

- What will be the contents of the registers after running the following RISC-V instructions:

.data

value1: .word 0x12345678

value2: .word 0x12f4f688

.text

la x10, value1

lw x11, 0(x10)

lh x12, 0(x10)

lh x13, 2(x10)

lb x14, 0(x10)

lb x15, 1(x10)

lb x16, 2(x10)

lb x17, 3(x10)

lb x18, 4(x10)

la x10, value2

lw x11, 0(x10)

lh x12, 0(x10)

lh x13, 2(x10)

lb x14, 0(x10)

lb x15, 1(x10)

lb x16, 2(x10)

lb x17, 3(x10)

lb x18, 4(x10)

Load and store instructions.

Assume RV32. Any location not written stores 0.

- What will be the contents of the registers after running the following RISC-V instructions:

.data

value1: .word 0x12345678

value2: .word 0x12f4f688

.text

la x10, value

lw x11, 0(x10)

x11 <- 0x12345678

lh x12, 0(x10)

x11 <- 0x00005678

lh x13, 2(x10)

x11 <- 0x00001234

lb x14, 0(x10)

x11 <- 0x00000078

lb x15, 1(x10)

x11 <- 0x00000056

lb x16, 2(x10)

x11 <- 0x00000034

lb x17, 3(x10)

x11 <- 0x00000012

lb x18, 4(x10)

x11 <- 0x00000000

Load and store instructions.

- What will be the contents of the registers after running the following RISC-V instructions:

```
.data
```

```
value1: .word 0x12345678
```

```
value2: .word 0x12f4f688
```

```
.text
```

```
la x10, value2
```

```
lw x11, 0(x10)
```

```
# x11 <- 0x12f4f688
```

```
lh x12, 0(x10)
```

```
# x12 <- 0xfffff688
```

```
lh x13, 2(x10)
```

```
# x13 <- 0x000012f4
```

```
lb x14, 0(x10)
```

```
# x14 <- 0xffffffff88
```

```
lb x15, 1(x10)
```

```
# x15 <- 0xffffffff6
```

```
lb x16, 2(x10)
```

```
# x16 <- 0xffffffff4
```

```
lb x17, 3(x10)
```

```
# x17 <- 0x00000012
```

```
lb x18, 4(x10)
```

```
# x18 <- 0x00000000
```

Load and Store instructions.

- Write the corresponding RISC-V (RV32) instructions for the following C-codes:
 - Assuming single-precision elements. X15 <- Base pointer, x12 <- h, x11 <- g.

```
int A[100];  
g = h + A[3];
```

* Need to calculate the offset to get the values of A[3].

Load and Store instructions.

- Write the corresponding RISC-V (RV32) instructions for the following C-codes:
 - Assuming single-precision elements. X15 <- Base pointer, x12 <- h, x11 <- g.

```
int A[100];  
g = h + A[3];
```

```
lw x10, 12(x15)  
add x11, x12, x10
```

- Need to calculate the offset to get the values of A[3].
 - The offset address is a function of the size of an element in an array.

Load and Store instructions.

- Write the corresponding RISC-V (RV32) instructions for the following C-codes:
 - Assuming single-precision elements. X15 <- Base pointer and x12 <- h.

```
int A[100];  
A[10] = h + A[3];
```

* Two offsets need to be calculated. What is special about the offset calculation in the 'store' instruction?

Load and Store instructions.

- Write the corresponding RISC-V (RV32) instructions for the following C-codes:
 - Assuming single-precision elements. X15 <- Base pointer and x12 <- h.

<code>int A[100];</code>	<code>lw x10, 12(x15)</code>
<code>A[10] = h + A[3];</code>	<code>add x10, x10, x12</code>
	<code>sw x10, 40(x15)</code>

* Two offsets need to be calculated. What is special about the offset calculation in the 'store' instruction?

- The offset of a store operation should always be aligned.
 - The offset must be a multiples of 4 (RV32) and 8(RV64).

Load and store instructions.

- Assume we have an integer array of numbers whose base address is in x11. How do we load the following parts of the first element in RISC-V:
 - Lower half
 - Upper half
 - Least significant byte
 - Most significant byte.

Load and store instructions.

- Assume we have an integer array of numbers whose base address is in x11. How do we load the following parts of the first element in RISC-V:

- Lower half -> lh x10, 0(x11)
- Upper half -> lh x10, 2(x11)
- Least significant byte -> lb x10, 0(x11)
- Most significant byte. -> lb x10, 3(x11)

Solved example

1. A register x19 in RV64 stores the value AB BC CD DE EF F1 12 23. Let x20=1024. Show the memory state after implementing
- (i) sd x19, 0(x20)
 - (ii) sh x19, 0(x20)

Byte Address:	Contents:
1024	23
1025	12
1026	F1
1027	EF
1028	DE
1029	CD
1030	BC
1031	AB

Byte Address:	Contents:
1024	23
1025	12

Tutorial on Logical Instructions

Sparsh Mittal

This presentation assumes that learner is familiar with 2's complement representation.

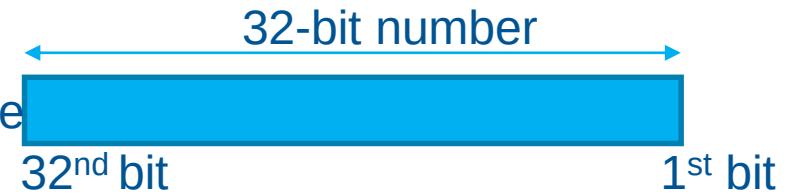
Multiplication without mul

Write a program to multiply register a0 by 68, without using mul instruction. Store Result in a3

```
Slli a1 a0 6      # y = x*64  
Slli a2 a0 2      # z = x*4  
Add a3 a1 a2      #v = y+z = x*68
```

Solved Question

How counting is done



Read 15th bit of register a0 and copy it as LSB of a1

```
li a1 0b10000000000000000 #1 followed by 14 zeros
and a1 a1 a0
srli a1 a1 14
```

```
andi a1 a0 0b10000000000000000
srli a1 a1 14
```

```
lui a1 0x4 #1 followed by 14 zeros
and a1 a1 a0
srli a1 a1 14
```


Create a mask
Store 1 at 26th bit of a0. Remaining bits are zero.

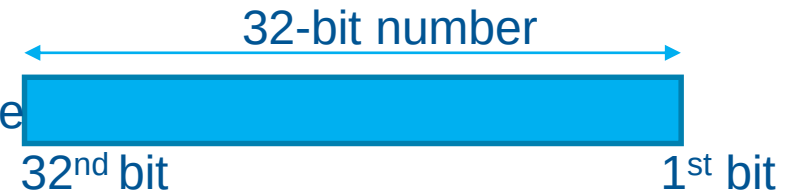
```
Li a0 0b10 0000 0000 0000 0000 0000  
0000
```

```
Li a0 1  
Slli a0 a0 25
```

```
Li a0 0x2000000
```

Solved Question

How counting is done

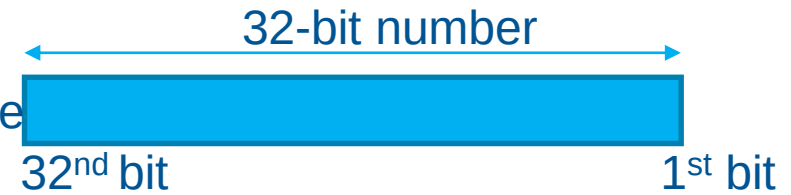


Write code to

- (i) reset 25th bit,
- (ii) set 29th bit
- (iii) Flip 21st bit

Solved Question

How counting is done



- Assume RV32. Register a0 currently stores some non-zero value.

Set its 29th bit to 1.

```
li a1 1
slli a2 a1 28
or a0 a0 a2
```

C code

```
int a;
//a="some value";
a = a | 0x100000000;
```

Reset its 29th bit

```
li a1 1
slli a2 a1 28
not a3 a2
and a0 a0 a3
```

C code

```
int a;
//a="some value";
a = a & 0xefffffff;
```

Flip its 29th bit

```
li a1 1
slli a2 a1 28
xor a0 a0 a2
```

C code

```
int a;
//a="some value";
a = a^0x100000000;
```

Solved Question

- Assume RV32. Check whether the lowest 4 bits of register a0 are 1010. If so, put 1 in a5
- li a1 0b1010
- li a2 0b1111
- and a3 a0 a2
- xor a4 a3 a1 # if a3 was 1010, then a4 should be all zeros.
- Seqz a5 a4

Solved Question [Case 1: 1010 is there]

- Assume RV32. Check whether the lowest 4 bits of register a0 are 1010. If so, put 1 in a5
 - a0 = xxxx xxxx xxxx xxxx xxxx xxxx xxxx 1010
- li a1 0b1010
 - a1 = 0000 0000 0000 0000 0000 0000 0000 1010
- li a2 0b1111
 - a2 = 0000 0000 0000 0000 0000 0000 0000 1111
- and a3 a0 a2
 - a3 = 0000 0000 0000 0000 0000 0000 0000 1010
- xor a4 a3 a1 # if a3 was 1010, then a4 should be all zeros.
 - a4 = 0000 0000 0000 0000 0000 0000 0000 0000
- Seqz a5 a4
 - a5 = 0000 0000 0000 0000 0000 0000 0000 0001 a5 becomes 1

Solved Question [Case 2: 1010 is not there]

- Assume RV32. Check whether the lowest 4 bits of register a0 are 1010. If so, put 1 in a5
 - a0 = xxxx xxxx xxxx xxxx xxxx xxxx xxxx 1110
- li a1 0b1010
 - a1 = 0000 0000 0000 0000 0000 0000 0000 1010
- li a2 0b1111
 - a2 = 0000 0000 0000 0000 0000 0000 0000 1111
- and a3 a0 a2
 - a3 = 0000 0000 0000 0000 0000 0000 0000 1110
- xor a4 a3 a1 # if a3 was 1010, then a4 should be all zeros.
 - a4 = 0000 0000 0000 0000 0000 0000 0000 0100
- Seqz a5 a4
 - a5 = 0000 0000 0000 0000 0000 0000 0000 0000 a5 becomes 0

Solved Question [second way of writing code]

- Assume RV32. Check whether the lowest 4 bits of register a0 are 1010. If so, put 1 in a5
- li a1 0b1010
- li a2 0b1111
- and a3 a0 a2
- beq a3 a1 L1
- j Exit
- L1: li a5 1
- Exit: NOP

Solved example

	C code	RISC-V code
Left-shift by 2 = Multiplication by 4	<pre>a=64; b=2; c = a<<b; /*Final value of c becomes 256 */</pre>	
Right-shift by 2 = Division by 4	<pre>a=64; b=2; c = a>>b; /*Final value of c becomes 16 */</pre>	
Right-shift by 2 = Division by 4. Floor(50/4) = 12	<pre>a=50; b=2; c = a>>b; /*Final value of c becomes 12 */</pre>	

Solved example

	C code	RISC-V code
Left-shift by 2 = Multiplication by 4	<pre>a=64; b=2; c = a<<b; /*Final value of c becomes 256 */</pre>	<pre>li a0, 64 li a1, 2 sll a2, a0, a1</pre>
Right-shift by 2 = Division by 4	<pre>a=64; b=2; c = a>>b; /*Final value of c becomes 16 */</pre>	<pre>li a0, 64 li a1, 2 sra a2, a0, a1</pre>
Right-shift by 2 = Division by 4. Floor(50/4) = 12	<pre>a=50; b=2; c = a>>b; /*Final value of c becomes 12 */</pre>	<pre>li a0, 50 li a1, 2 sra a2, a0, a1</pre>

Branch and For/While Loops

RISC-V

“if condition” with equality

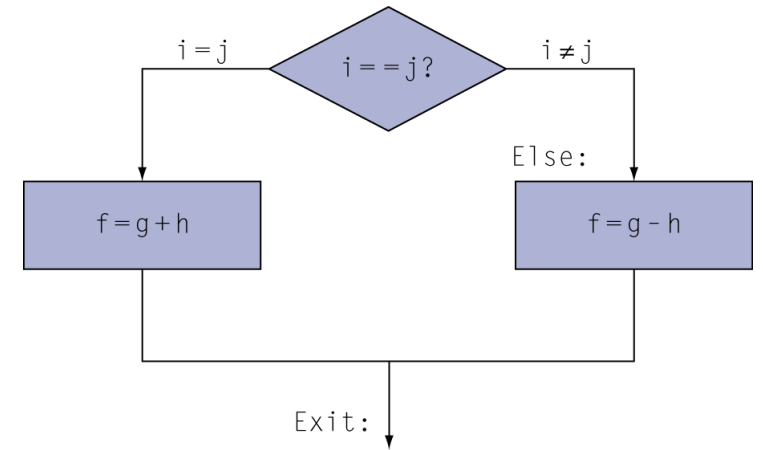
- C code:

```
if (i==j) f = g+h;  
else f = g-h;
```

Write RISC-V code for following mapping

- $f \rightarrow x19$ $g \rightarrow x20$
- $h \rightarrow x21$ $i \rightarrow x22, j \rightarrow x23$
- Compiled RISC-V code:

```
        bne x22, x23, Else  
        add x19, x20, x21  
        beq x0,x0,Exit // unconditional  
Else:   sub x19, x20, x21  
Exit:   ...
```



“if condition” with greater than

- Let variables f, g, h, i, and j be assigned to registers x5, x6, x7, x28, and x29, respectively. Write RISC-V code for this C code.

```
if(i > j)
    f = g-h
```

Solution: RISC-V has blt, beq, bne and bge instructions. We first express our C-code in a way that can be directly translated into RISC-V codes. This can be done in (at least) three ways:

“if condition” with greater than

```
if(i > j)
    f = g-h
```

Rewritten code 1

Rewritten code 2

Rewritten code 3

C code

```
if(i<j) goto EXIT;
if(i==j) goto EXIT;
f=g-h
EXIT:
```

```
if (j>=i) goto EXIT;
f=g-h
EXIT:
```

```
if (j<i)
f=g-h
```

RISC-V

```
blt x28, x29, EXIT
beq x28, x29, EXIT
sub x5, x6, x7
EXIT: NOP
```

```
bge x29, x28, EXIT
sub x5, x6, x7
EXIT: NOP
```

```
blt x29, x28, L1
j EXIT
L1: sub x5, x6, x7
EXIT: NOP
```

Solved Example

- Convert below code to RISC-V. $x/y/z$ are $a0/a1/a2$.

if ($x==y$ or $x>z$)

$z=50;$

```
        beq a0, a1, L1
        bgt a0, a2, L1
        j exit
L1: li a2, 50
exit:
        nop
```

Convert to RISC-V

- Convert `k -= i>1? 5: 3;`

This is equal to
`if(i>1) k= k-5;`
`else k = k-3;`

```
li x10, 1                # load '1' for comparison
    blt x10, x22, cond1   # check condition i>1 and branch on True
    beq x0, x0, cond2     # unconditional branch taken on previous being False
cond1:
    addi x24, x24, -5      # subtract '5' from K
    beq x0, x0, exit      # exit
cond2:
    addi x24, x24, -3      # subtract '3' from k
    beq x0, x0, exit      # exit
exit: NOP
```

Solved Example

- Convert below code to RISC-V. x is in a0.

switch(x)

{case 2:

y= 5;

break;

case 3:

y=7;

case 6:

y=9;

default:

y=100;

}

```
li a2, 2
li a3, 3
li a4, 6
beq a0, a2, L1 #case 2
beq a0, a3, L2 #case 3
beq a0, a4, L3 # case 6
j default
L1:
li a1, 5 #y=5
j exit #break
L2:
li a1, 7
L3:
li a1, 9
default:
li a1, 100
exit:
nop
```


Decision making (Conditional operations)

- Conditional vs Unconditional branches:

- beq reg1, reg2, L1(symbolic address) vs. j L1

- Example 'if-statement':

- f <- x10, g <- x11, h <- 11, h <- x12, l <- x13, j <- x14

- C-code:

```
if (i==j):  
    f = g + h;
```

- Can be solved in two ways: “beq and bne”. Can you guess which one is more efficient?

Decision making (Conditional operations)

- Conditional vs Unconditional branches:
 - beq reg1, reg2, L1(symbolic address) vs. j L1
- Example 'if-statement':
 - `f <- x10, g <- x11, h <- 11, h <- x12, l <- x13, j <- x14`
 - C-code:

```
if (i==j):  
    f = g + h;
```

- Can be solved in two ways: “beq and bne”. Can you guess which one is more efficient?
- * **'bne' instruction is more efficient.**

Decision making (Conditional operations)

RISC-V code:

```
    beq x13, x14, SUM
    j exit
SUM:
    add x10, x11, x12
EXIT:
    NOP
```

RISC-V code:

```
    bne x13, x14, EXIT
    add x10, x11, x12
EXIT:
    NOP
```

Decision making (Conditional operations)

- What makes 'blt' different than 'bltu'?

- blt reg1, reg2, label

- if (reg1<reg2)

- goto label;

- bltu reg1, reg2, label

- if (|reg1|<|reg2|) # sign is ignored, absolute mag. only

- goto label;

Decision making (Conditional operations)

- Three types of loops:

- While

- Do-while

- For

- * How would you implement these functions using RISC-V branching instructions?

Solved Question: Convert “for” loop code

- Variables i in x28, j in x29. Base address of “long int” array A in x10. sum in x11.

for(int i=0; i<j; i++)

sum = sum+A[i];

Let us break it into simple operations in C/C++

C/C++ code

i=0

LOOP: if (i>=j) goto EXIT

offset = i*8

addr = A+offset

val = Memory[addr]

sum = sum+val

i= i+1

goto LOOP

EXIT:

RISC-V code

addi x28, x0, 0

LOOP: bge x28, x29, EXIT

slli x12, x28, 3

add x12, x12, x10

ld x7, 0(x12)

add x11, x11, x7

addi x28, x28, 1

beq x0, x0, LOOP

EXIT:

Solved Question: Convert “for” loop code

- Variables i in x28, j in x29. sum in x11.

for(int i=0; i<j; i++)

sum = sum+i;

Let us break it into simple operations in C/C++

C/C++ code

i=0

LOOP: if (i>=j) goto EXIT

sum = sum+i

i= i+1

goto LOOP

EXIT:

RISC-V code

addi x28, x0, 0

LOOP: bge x28, x29, EXIT

add x11, x11, x28

addi x28, x28, 1

beq x0, x0, LOOP

EXIT:

Examples RISC-V codes:

Assume we have an array in memory that contains *int* arr = {1,2,3,4,5,6,0}*. Let register *s0* hold the *address* of the element at *index 0 in arr*. You may assume integers are *four-bytes* and our values are word-aligned. What do the snippets of RISC-V code do? Assume that all the instructions are run one after the other in the same context.

a) lw t0, 12(s0)

b) sw t0, 16(s0)

c) slli t1, t0, 2
add t2, s0, t1
lw t3, 0(t2)
addi t3, t3, 1
sw t3, 0(t2)

d) lw t0, 0(s0)
xori t0, t0, -1
addi t0, t0, 1

Examples RISC-V codes:

Assume we have an array in memory that contains *int* arr = {1,2,3,4,5,6,0}*. Let register *s0* hold the *address* of the element at *index 0 in arr*. You may assume integers are *four-bytes* and our values are word-aligned. What do the snippets of RISC-V code do? Assume that all the instructions are run one after the other in the same context.

a) lw t0, 12(s0)

*Sets t0 equal to arr[3]
X = arr[3];*

b) sw t0, 16(s0)

*Stores t0 into arr[4]
Arr[4] = arr[3]*

c) slli t1, t0, 2 //t0*4
 add t2, s0, t1 //add this to base-address
 lw t3, 0(t2)
 addi t3, t3, 1
 sw t3, 0(t2)

Increments arr[t0] by 1

d) lw t0, 0(s0)
 xori t0, t0, -1 //takes 1's complement
 addi t0, t0, 1 //takes 2's complement

*Sets t0 to -1 * arr[0]*